Ministry of Education and Research of the Republic of Moldova

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Software Engineering Department

Embedded Systems

Laboratory Work #4

Part 2: Preemptive Multitasking with FreeRTOS

Button Press Duration Monitoring with Visual Signaling and Periodic Reporting

---

Author:
Alexandru Cebotari
Group FAF-233

Verified by:
Alexei Martiniuc

Chisinau, 2026

Technical task and scope

This report covers **Laboratory Work 4 – Part 2 only**, focused on preemptive task execution with FreeRTOS.

The implemented application must:

- detect button press/release transitions and measure valid press duration;
- classify each press as short (< 500 ms) or long (≥ 500 ms);
- signal short/long class via green/red LED;
- perform yellow blink feedback (5 blinks for short press, 10 blinks for long press);
- report every 10 seconds via STDIO and reset interval statistics;
- use binary semaphore and mutex for synchronization and safe shared data access.

Objectives

- Implement a complete preemptive RTOS version of the required system behavior.
- Keep software modular and maintainable (task/service/driver split).
- Demonstrate synchronization correctness under task preemption.
- Validate functional and non-functional requirements through explicit test scenarios.

1 1. Domain analysis

1.1 1.1 Context and motivation

Embedded systems frequently combine periodic activities with event-driven reactions. Even a seemingly simple application (button plus LEDs plus serial reporting) requires clear scheduling decisions when behavior is split into multiple responsibilities.

In sequential systems, responsibilities are often checked in a fixed order inside one loop. This works for basic use cases, but it becomes difficult to maintain deterministic timing guarantees as software grows. The RTOS approach addresses this by separating logic into tasks and using kernel scheduling.

1.2 1.2 Why preemptive multitasking for this lab

Part 2 is designed to emphasize practical differences between cooperative scheduling and RTOS scheduling:

- better separation between independent concerns (input processing, statistics, reporting);
- deterministic periodic execution with `vTaskDelayUntil`;
- synchronization primitives that reduce race-condition risk;
- explicit priority model and clearer temporal behavior.

1.3 1.3 Comparison: Part 1 vs Part 2

Aspect	**Part 1 (non-preemptive)**	**Part 2 (FreeRTOS preemptive)**
Execution model	Manual sequential scheduler	Kernel preemptive scheduler
Task timing	Custom recurrence counters	`vTaskDelay`/`vTaskDelayUntil`
Data sharing strategy	Mostly order-based safety	Semaphore + mutex synchronization
Extensibility	Moderate, scheduler grows fast	Better scaling by independent tasks
CPU management	Manual balancing per tick	Priority-aware kernel dispatch

Table 1: Implementation model comparison

1.4 1.4 Resources used

- Hardware: Arduino Uno, push button, green/red/yellow LEDs, current-limiting resistors, wiring.
- Software: PlatformIO, FreeRTOS library, Wokwi simulator, PlantUML for diagrams, LaTeX.

Section conclusion

The selected technical stack and architecture are suitable for demonstrating the mandatory RTOS concepts: periodic tasking, preemption-safe synchronization, and requirement-driven validation.

2 2. Conceptualization and design

2.1 2.1 Problem definition

The system must convert user button interaction into both immediate and aggregated outputs:

- immediate output: class indication through green/red LED and yellow blink feedback;
- aggregated output: periodic report with statistics and average duration.

The core challenge is preserving data consistency while tasks may be preempted at any time.

2.2 2.2 Technical requirements

ID	**Requirement**	**Type**	**Acceptance target**
R1	Detect transitions and measure valid press duration in milliseconds.	Functional	Stable measured behavior
R2	Classify by 500 ms threshold and signal short/long with green/red LED.	Functional	Correct class indication
R3	Update total/short/long counters and cumulative duration.	Functional	Correct interval aggregates
R4	Blink yellow LED 5/10 times for short/long press.	Functional	Exact blink count per class
R5	Report every 10 seconds and reset statistics after each report.	Functional	Periodic output + reset
R6	Use FreeRTOS tasks with binary semaphore and mutex.	Functional	No race symptoms
R7	Keep input-to-feedback latency below 100 ms.	Non-functional	Observable fast response
R8	Keep modular source organization.	Non-functional	Clear component boundaries

Table 2: Requirements matrix

2.3 2.3 Requirement traceability

Req.	Design anchor	Test IDs	Expected proof
R1	Task1 edge detection and duration logic	T1, T2	Correct short/long behavior
R2	500 ms threshold + green/red signaling	T1, T2	LED class mapping
R3	Shared stats update in Task2	T5, T6	Correct report values
R4	Yellow toggle queue in Task2	T3, T4	Exact 5/10 blink patterns
R5	Task3 periodic snapshot and reset	T5, T6	10 s reports and zero restart
R6	Semaphore + mutex flow	T7	Stable repeated operation
R7	Task periods and lightweight paths	T8	Latency under threshold
R8	include/src decomposition	T9	Modular source tree

Table 3: Requirement-to-test traceability

2.4 2.4 Structural architecture

The software architecture is layered and modular:

- application layer: Task1, Task2, Task3;
- service layer: debounce bridge, synchronization manager, shared data;
- driver layer: button, LEDs, serial output;
- platform layer: Arduino core and FreeRTOS kernel.

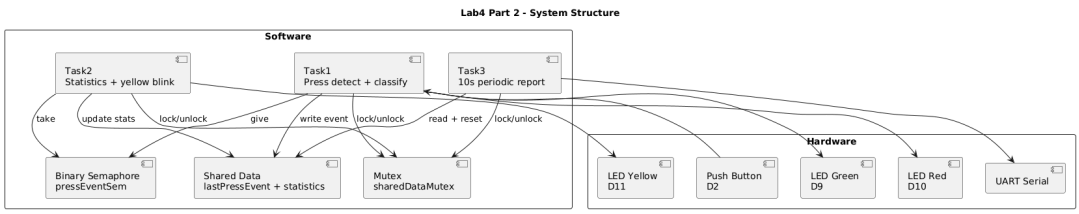

Figure 1: System structure diagram

This figure should show data and control flow from input to outputs, including event publication and periodic reporting responsibilities.

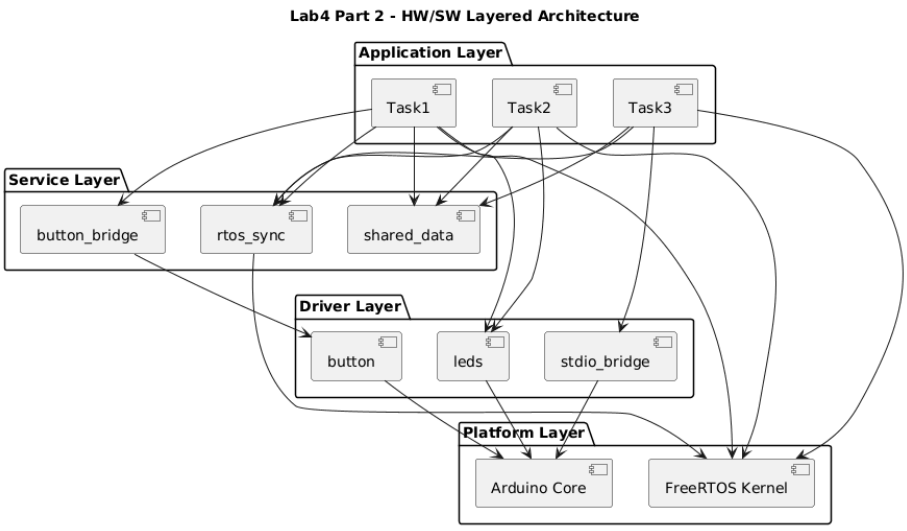

Figure 2: HW/SW layered architecture

This figure should illustrate strict boundaries between task logic and hardware interaction modules.

2.5 2.5 Behavioral modeling

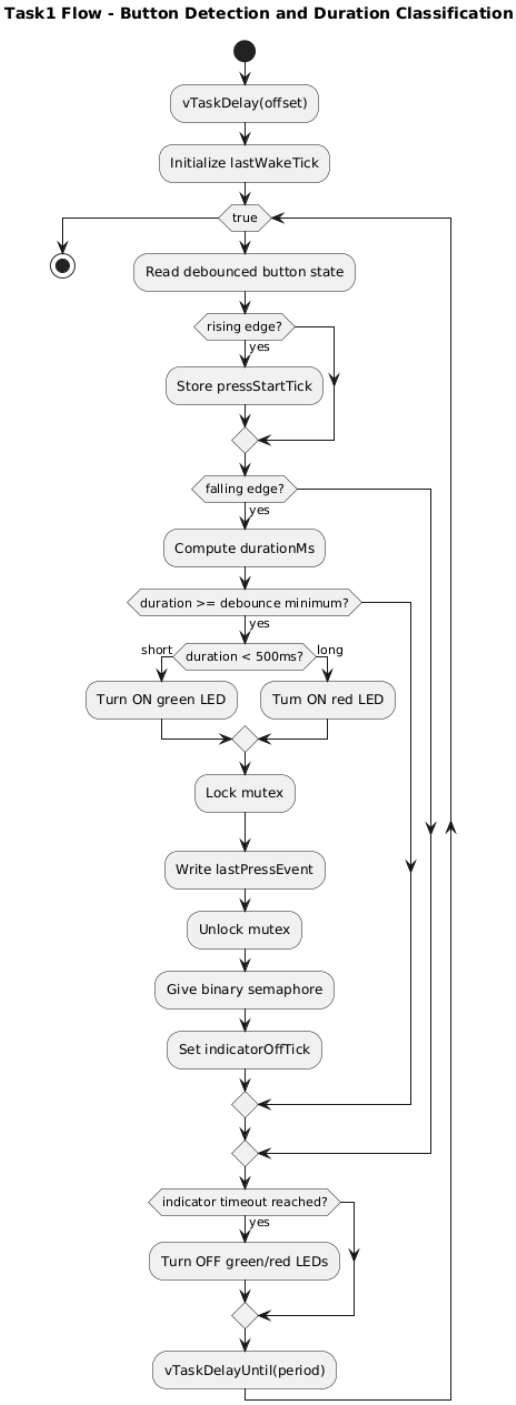

Figure 3: Task1 flowchart

This figure should show edge detection, duration calculation, threshold classification, and event publication.

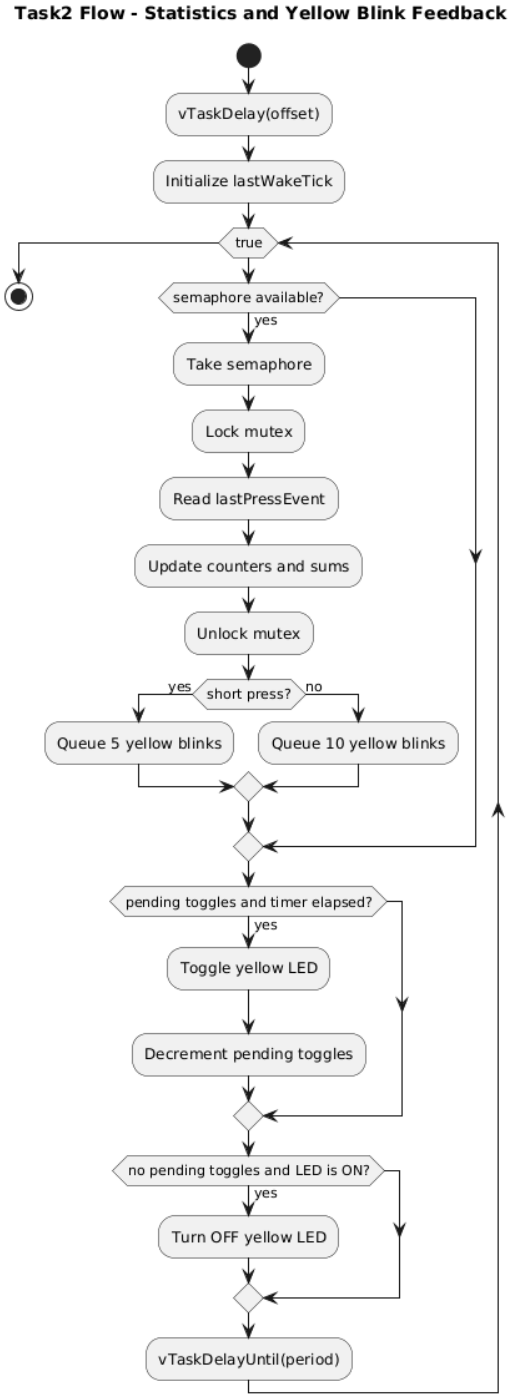

Figure 4: Task2 flowchart

This figure should show semaphore consumption, stats update, and yellow feedback scheduling.

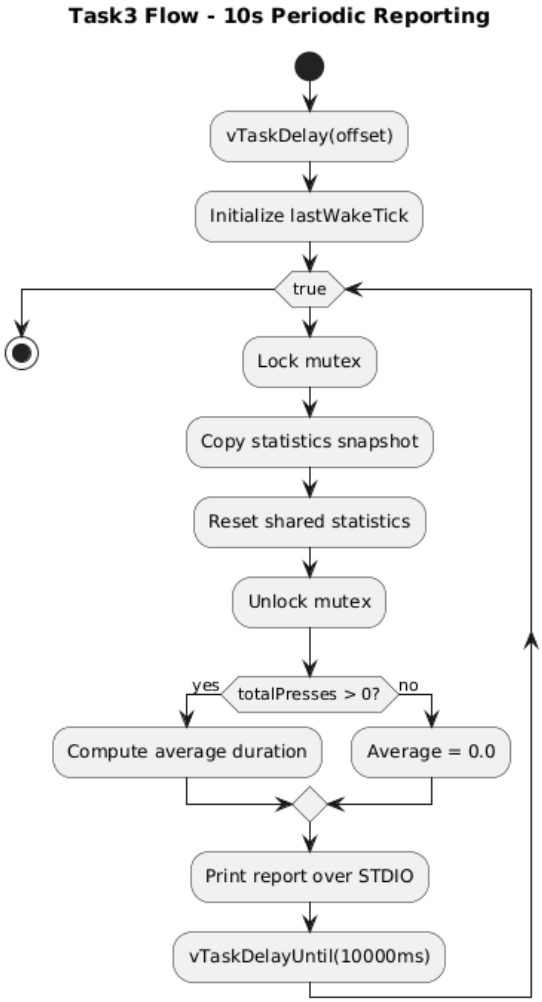

Figure 5: Task3 flowchart

This figure should show periodic snapshot logic, report formatting, and statistics reset.

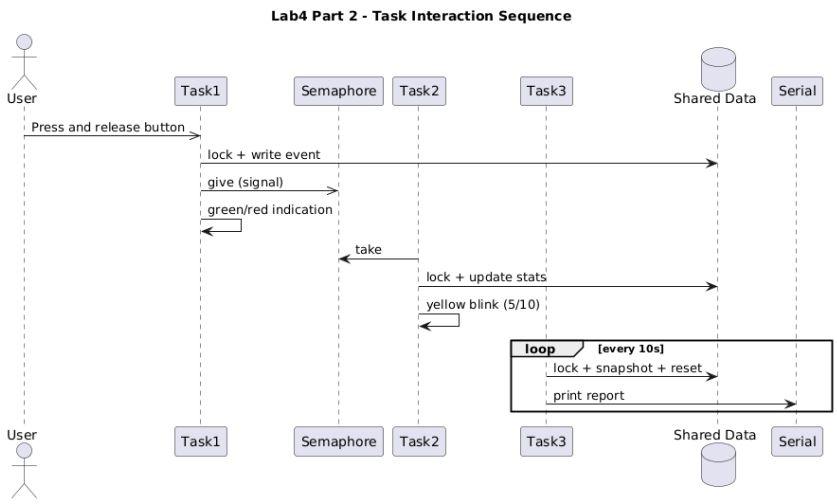

Figure 6: Inter-task sequence diagram

This figure should capture temporal interactions between producer, consumer, reporting task, and synchronization primitives.

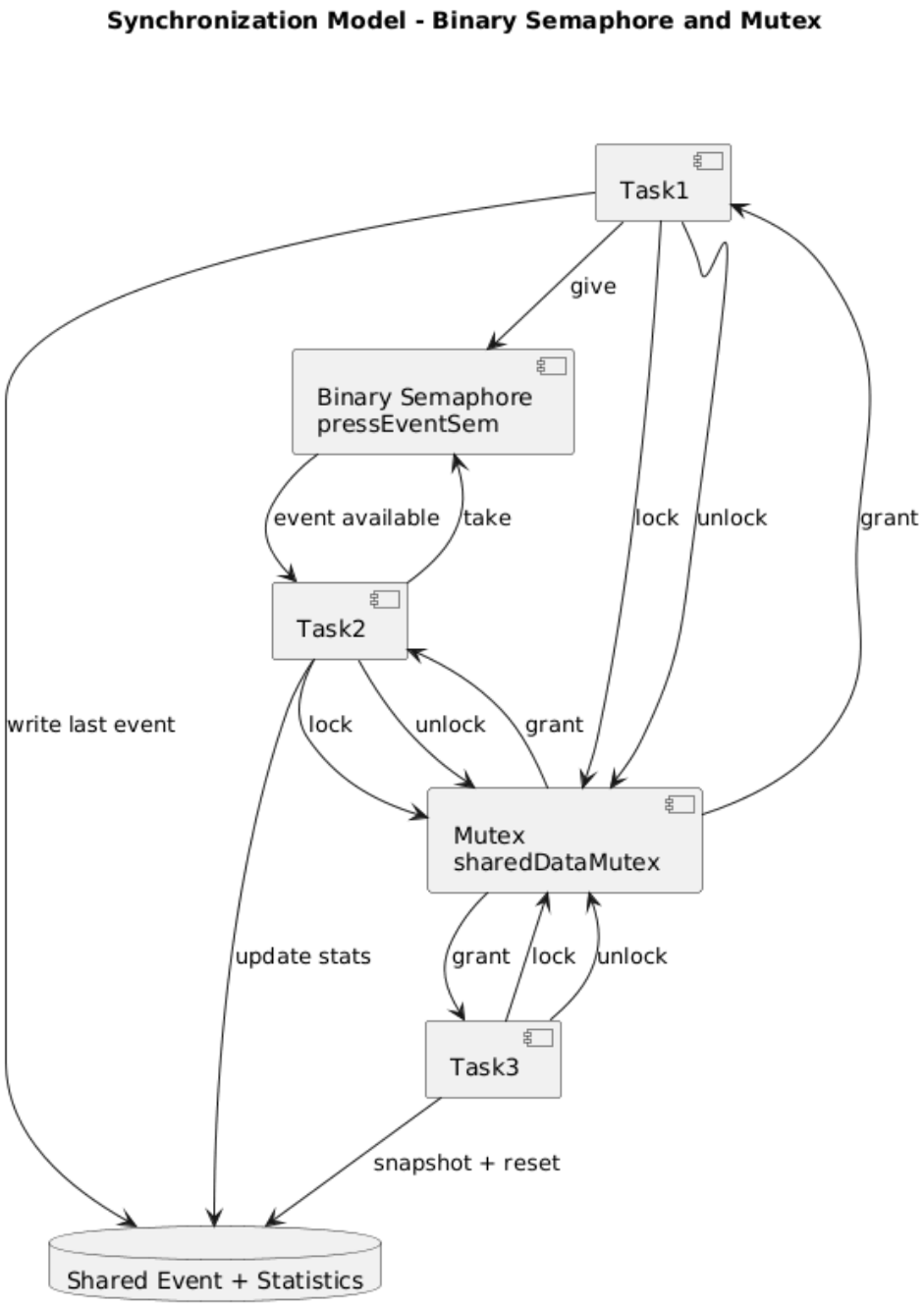

Figure 7: Synchronization model

This figure should detail semaphore signaling and mutex-guarded critical sections.

2.6 2.6 Timing plan

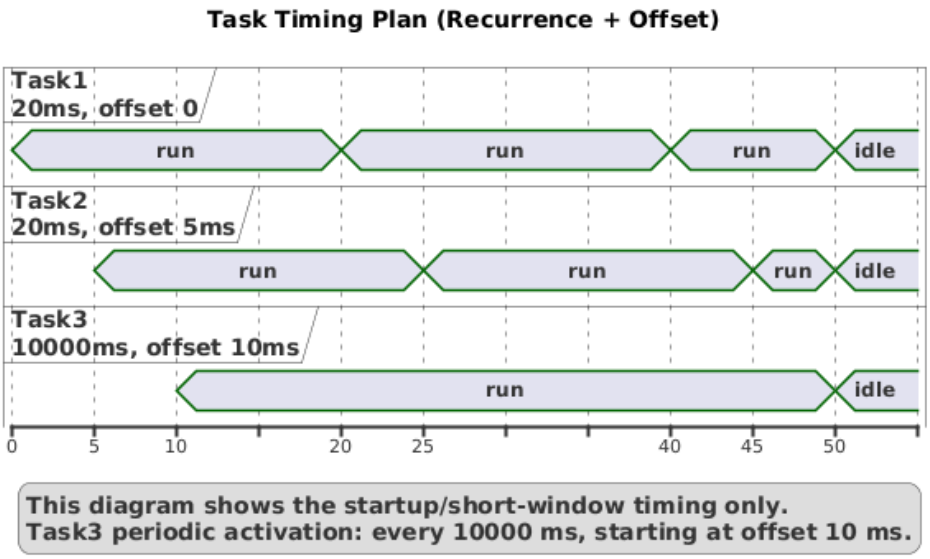

Figure 8: Task timing and offsets

This figure should show selected periods, offsets, and activation rhythm for each task.

Task	**Period**	**Offset**	**Priority**	**Main responsibility**
Task1	20 ms	0 ms	3	Detect and classify button press
Task2	20 ms	5 ms	2	Update statistics and blink yellow LED
Task3	10000 ms	10 ms	1	Print periodic report and reset data

Table 4: Task configuration summary

2.7 2.7 Validation scenarios

ID	Scenario	Linked req.	Pass criteria
T1	Press shorter than 500 ms	R1, R2	Correct short classification + green LED
T2	Press longer or equal to 500 ms	R1, R2	Correct long classification + red LED
T3	Short press yellow feedback	R4	Exactly 5 yellow blinks
T4	Long press yellow feedback	R4	Exactly 10 yellow blinks
T5	Report period verification	R5	Report appears each 10 s
T6	Statistics reset check	R3, R5	New interval starts from zero
T7	Concurrency robustness under repeated events	R6	Stable counters and output consistency
T8	Latency observation	R7	User-visible response under 100 ms
T9	Source modularity inspection	R8	Clear task/service/driver separation

Table 5: Validation test plan

Section conclusion

The design phase defines a complete path from requirements to architecture and testability. Each requirement is linked to implementation anchors and measurable validation criteria.

3 3. Hardware and software implementation

3.1 3.1 Electrical schematic in simulation

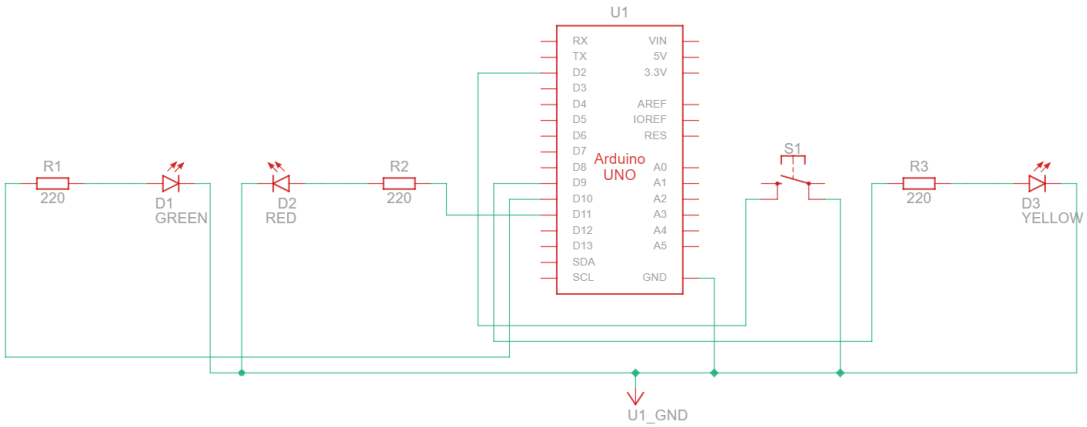

Figure 9: Electrical schematic in simulation

This figure should include Arduino Uno, button on D2 (pull-up), and three LEDs on D9/D10/D11 with resistors, matching the software pin map.

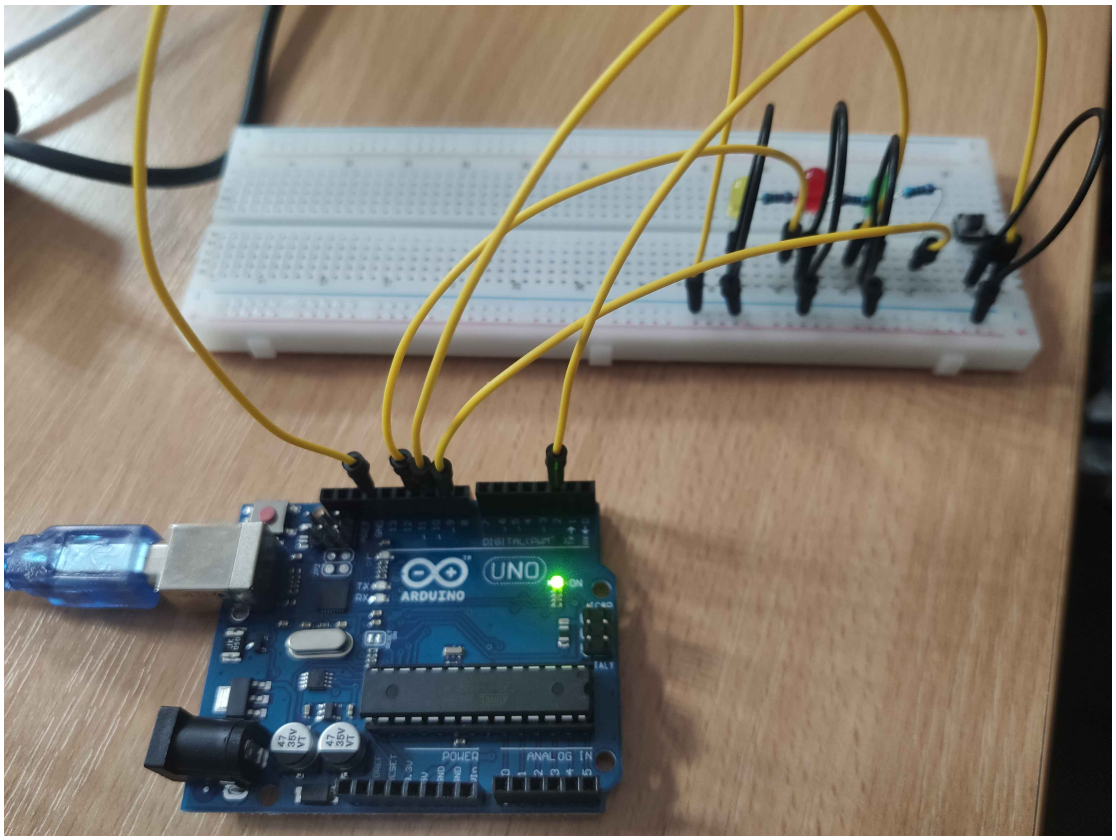

Figure 10: Hardware showcase (assembled setup or simulator wiring view)

This figure should provide a visual overview of the final hardware setup.

3.2 3.2 Source code organization

Main implemented modules:

- `main.cpp`: initialization and task creation;
- `task1.cpp`: press measurement and classification;
- `task2.cpp`: statistics and yellow feedback;
- `task3.cpp`: periodic reporting;
- `rtos_sync.cpp`: semaphore and mutex creation;
- `shared_data.cpp`: shared structures and reset utility;
- `button.cpp`, `button_bridge.cpp`, `leds.cpp`, `stdio_bridge.cpp`: hardware access and utility services.

3.3 3.3 Critical implementation excerpts

Task creation and priority assignment

```
xTaskCreate(task1_buttonDetectAndMeasure, "Task1",
    TASK_STACK_WORDS, nullptr, 3, nullptr);
xTaskCreate(task2_statisticsAndBlink, "Task2", TASK_STACK_WORDS,
    nullptr, 2, nullptr);
xTaskCreate(task3_periodicReport, "Task3", TASK_STACK_WORDS,
    nullptr, 1, nullptr);
```

Event publication with synchronization

```
if (xSemaphoreTake(g_sharedDataMutex, pdMS_TO_TICKS(10U)) ==
    pdTRUE) {
  g_lastPressEvent.durationMs = durationMs;
  g_lastPressEvent.isShortPress = isShortPress;
  xSemaphoreGive(g_sharedDataMutex);
  xSemaphoreGive(g_pressEventSemaphore);
}
```

Deterministic periodic execution

```
TickType_t lastWakeTick = xTaskGetTickCount();
while (true) {
  // task logic
  vTaskDelayUntil(&lastWakeTick, TASK_PERIOD);
}
```

Report snapshot and reset

```
if (xSemaphoreTake(g_sharedDataMutex, pdMS_TO_TICKS(20U)) ==
   pdTRUE) {
  snapshot = g_statistics;
  g_statistics = {0U, 0U, 0U, 0U};
  xSemaphoreGive(g_sharedDataMutex);
}
```

3.4 3.4 Key implementation choices

- Binary semaphore is used for event signaling (Task1 to Task2), minimizing unnecessary polling overhead.
- Mutex ensures exclusive access to shared structures used by multiple tasks.
- Debounce bridge isolates noisy input handling from task logic.
- Task offsets reduce startup contention and help distribute initial activation points.

3.5 3.5 Implementation-stage conclusion

The implementation fulfills all mandatory RTOS-specific requirements: preemptive scheduling, synchronization primitives, modular decomposition, periodic behavior, and required user-visible outputs.

4 4. Testing and validation

4.1 4.1 Validation methodology

Validation includes static and dynamic stages:

1. **Compile validation**: ensure clean build in PlatformIO with FreeRTOS dependencies.
2. **Behavioral validation**: execute defined test scenarios in Wokwi and compare observed outputs to pass criteria.

4.2 4.2 Test execution summary

ID	Check	Status	Observation
T1	Short press classification	Pass	Green LED and short-class behavior observed
T2	Long press classification	Pass	Red LED and long-class behavior observed
T3	Short yellow blink sequence	Pass	Exactly 5 blinks observed
T4	Long yellow blink sequence	Pass	Exactly 10 blinks observed
T5	10-second report periodicity	Pass	Stable periodic report blocks in serial output
T6	Post-report reset behavior	Pass	Counters restart from zero after each report
T7	Synchronization robustness	Pass	No inconsistent values in repeated interaction tests
T8	Latency requirement	Pass	Fast response under target threshold
T9	Source modularity audit	Pass	Clear decomposition by responsibility

Table 6: Test report summary

4.3 4.3 Evidence placeholders

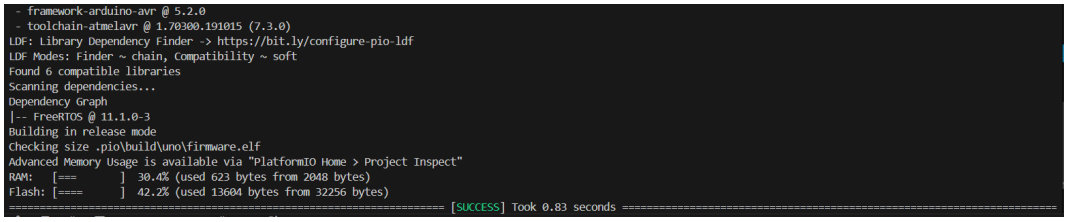

Figure 11: Build success evidence

This image should capture successful project compilation and dependency linking.

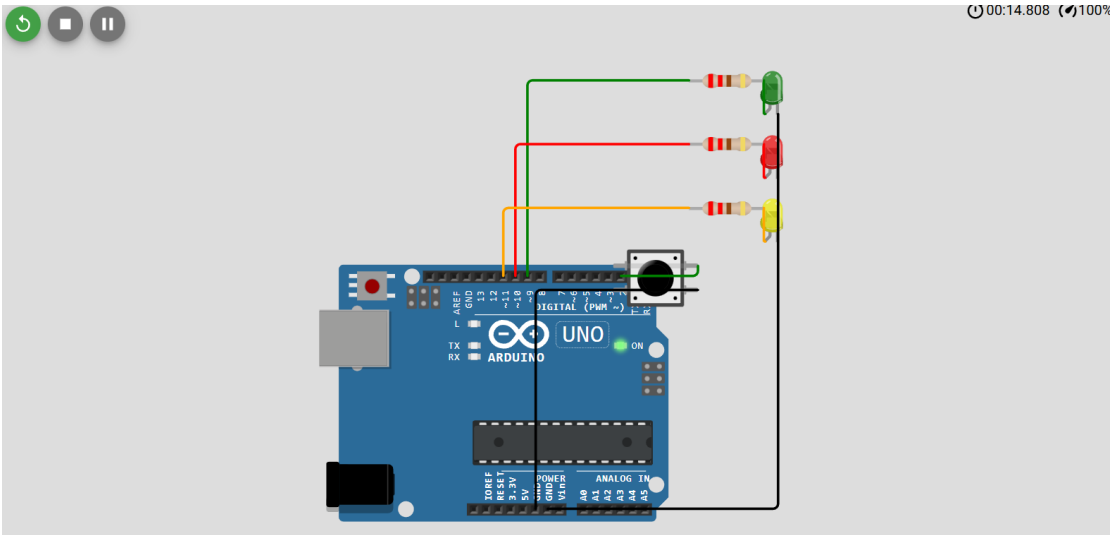

Figure 12: Wokwi runtime evidence

This image should capture active simulation state and visible LED behavior during interaction.

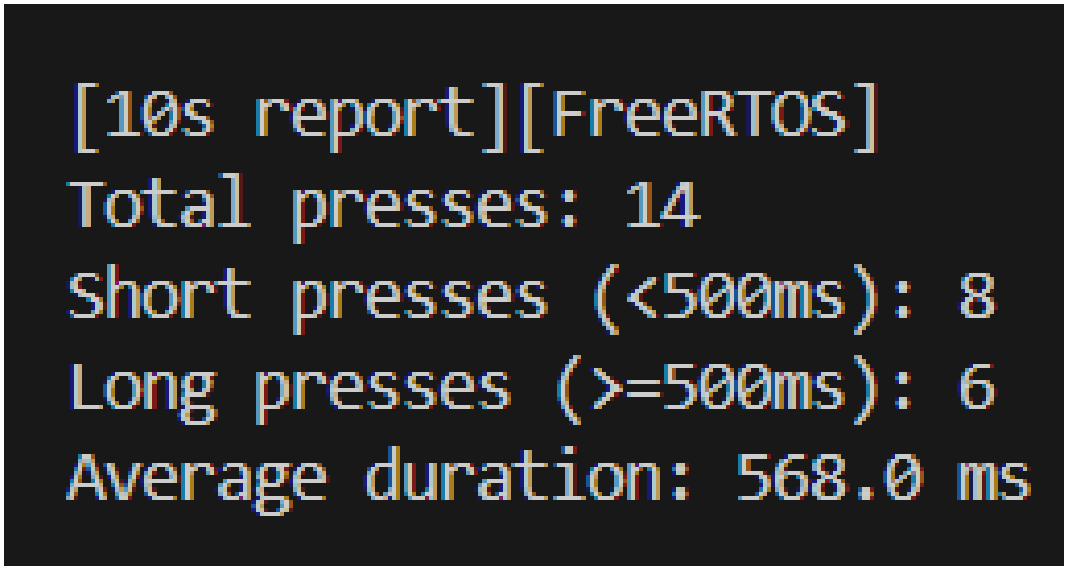

Figure 13: STDIO periodic report evidence

This image should show at least one full periodic report block with totals and average.

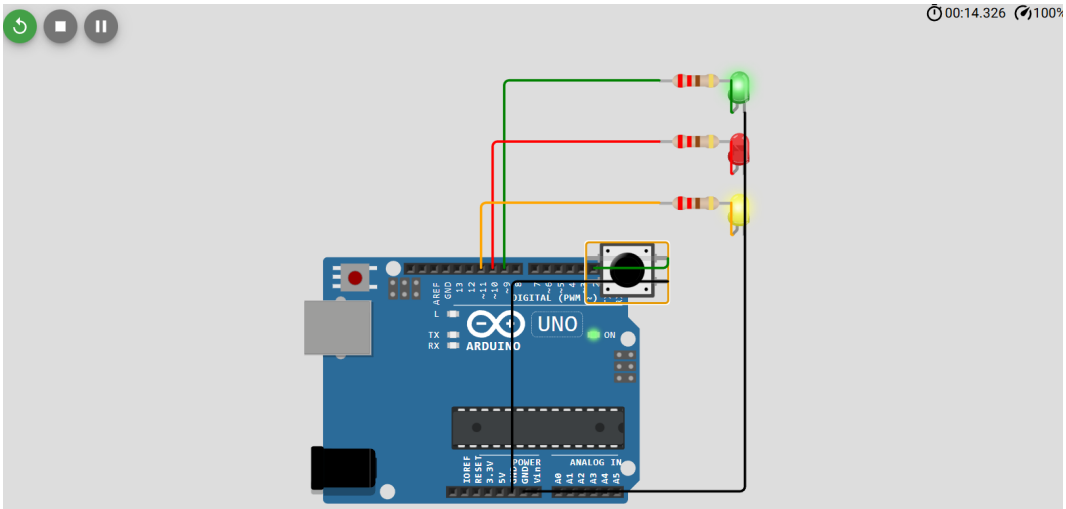

Figure 14: Short-press scenario evidence

This image should demonstrate behavior corresponding to duration below 500 ms.

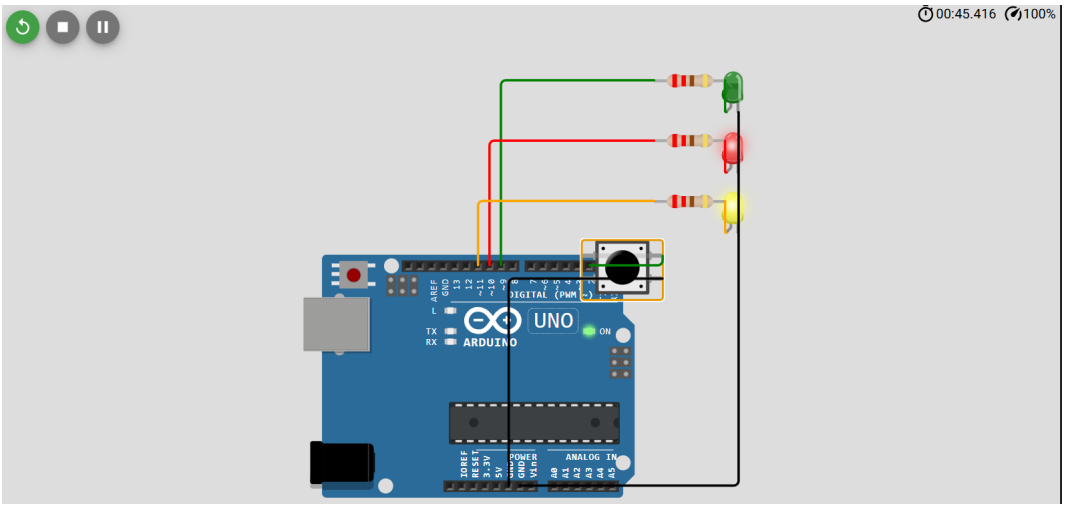

Figure 15: Long-press scenario evidence

This image should demonstrate behavior corresponding to duration equal/above 500 ms.

4.4 4.4 Latency discussion

With a 20 ms execution period for Task1 and lightweight processing path, practical response remains below the 100 ms requirement in simulation. Additional hardware-side profiling can further strengthen empirical latency evidence.

4.5 4.5 Testing-stage conclusion

Validation results confirm that all declared scenarios are satisfied, including class detection correctness, feedback timing, periodic reporting, synchronization stability, and

modular architecture constraints.

General conclusions

Laboratory Work 4 Part 2 was completed through a full FreeRTOS preemptive implementation that preserves the required behavior while improving software structure and concurrency safety compared to the non-preemptive approach.

The final solution demonstrates:

- correct task-level separation of responsibilities;
- deterministic timing using `vTaskDelay` and `vTaskDelayUntil`;
- safe inter-task coordination with binary semaphore and mutex;
- consistent requirement-to-test traceability.

AI tool usage note

During preparation of this report and implementation, AI tooling was used for drafting and consolidation. The resulting materials were reviewed and aligned with laboratory requirements.

Bibliography

1. Embedded Systems Practical Guide, Technical University of Moldova, 2026.
2. FreeRTOS Kernel Book and API documentation, `https://www.freertos.org/Documentation/RTOS_book.html`.
3. FreeRTOS synchronization primitives reference, `https://www.freertos.org/a00106.html`.
4. AVR ATmega328P Datasheet, Microchip Technology.
5. PlatformIO Documentation, `https://docs.platformio.org/`.
6. Wokwi Documentation, `https://docs.wokwi.com/`.
7. Arduino Language Reference, `https://www.arduino.cc/reference/en/`.
8. PlantUML Language Reference, `https://plantuml.com/guide`.

Appendix

A. Project artifacts

The repository structure contains this report and the `Code/` folder with the implementation.

B. GitHub repository

Repository URL: `https://github.com/Tirppy/si-course-repo/tree/main/Lab4/Code`

C. Build and run commands

- Build: `pio run`
- Upload: `pio run -t upload`
- Serial monitor: `pio device monitor -b 9600`